CSCI 432/532, Spring 2026

Homework 3

Due Monday, February 9 at 9am

Submission Requirements

- Type or clearly hand-write your solutions into a PDF format so that they are legible and professional. Submit your PDF on Canvas.
- Put each problem on a separate page and submit it correctly via the Gradescope interface.
- Do not submit your first draft. Type or clearly re-write your solutions for your final submission. If your submission is not legible, we will ask you to resubmit.

Collaboration and Resources

The *best* resources for completing the homework are (in no particular order):

- Your own notes from lectures and problem sessions, and/or lecture recordings.
- The lecture notes linked from the course website for the lecture day.
- The questions channel on Discord.
- Office hours.
- Discussions with classmates.

You may also access almost any resource in preparing your solution to the homework. The exception is that you may not seek answers to the problems on the homework directly, such as by pasting them into a GenAI tool, searching for solutions on a search engine, looking for them on Chegg or similar websites, or asking another person for the solution.

Additionally, you ***must***

- Write your solutions in your own words—this means that you should never be *copying* anything of substance from any source, and
- credit every resource you use, including GenAI tools, by providing links or full chat transcripts. If you use the provided LaTeX template, you can use the `Sources` environment for this. Ask if you need help!

> Remember, if you choose to use GenAI tools, not only must you provide a full transcript of your conversation (or a link to one), you must also use the following prompt (or some variation of it), and provide a full transcript of the conversation.

Prompt:

You are acting as a teaching assistant for an advanced undergraduate/graduate algorithms and computer science theory course.
Your role is not to provide full solutions or final answers.
Instead, you should act like we are having a conversation. Ask me a single question and let me respond. Avoid asking me many questions at once or giving me a lot of information at one time. Guide me using hints, leading questions, partial insights, and sanity checks. Help me debug my own proofs, algorithms, or reductions. Do not give a complete solution or full proofs. Be patient with me. Don’t give me details so that I can move on to the next part of a problem. Make sure that I understood before moving on.
The course uses Jeff Erickson’s Models of Computation lecture notes, so you should guide me to answer problems using the definitions, notation, and style from those notes.
Assume I am a serious student trying to learn, not to shortcut the assignment. I am attaching the assignment, so you should refuse to provide a complete solution to the problems listed. Of course, you can walk me through the "Solved Problems" listed at the end if I ask.

1. For each of the following languages over the alphabet $\Sigma = \{0, 1\}$, describe a DFA that accepts the language, and briefly describe the purpose of each state. You can describe your DFA using a drawing, using formal mathematical notation, or using a product construction.

   (a) All strings in 1^*01^* whose length is a multiple of 3.

   (b) All strings whose ninth-to-last symbol is 0, or equivalently, the set

   $$\{x0z : x, z \in \Sigma^* \text{ and } |z| = 8\}.$$

   Hint: don't try to draw the second one. Maybe take a look at the grading rubric to see your options for how to represent DFAs.

NFA/DFA rubric. 10 points =

+ 2 for an unambiguous description of a DFA or NFA, including the states set Q, the start state s, the accepting states A, and the transition function δ.
  - Drawings:
    * Use an arrow from nowhere to indicate the start state s.
    * Use doubled circles to indicate accepting states A.
    * If $A = \emptyset$, say so explicitly.
    * If your drawing omits a junk/trash/reject/hell state, say so explicitly.
    * Draw neatly! If we can't read your solution, we can't give you credit for it.
  - Text descriptions: You can describe the transition function either using a 2d array, using mathematical notation, or using an algorithm.
    * You must explicitly specify $\delta(q, a)$ for every state q and every symbol a.
    * If you are describing an NFA with ε-transitions, you must explicitly specify $\delta(q, \varepsilon)$ for every state q.
    * If you are describing a DFA, then every value $\delta(q, a)$ must be a single state.
    * If you are describing an NFA, then every value $\delta(q, a)$ must be a set of states.
    * In addition, if you are describing an NFA with ε-transitions, then every value $\delta(q, \varepsilon)$ must be a set of states.
  - Product constructions: You must give a complete description of each of the DFAs you are combining (as either drawings, text, or recursive products), together with the accepting states of the product DFA. In particular, we will not assume that product constructions compute intersections by default.

+ 4 for briefly explaining the purpose of each state in English. This is how you argue that your DFA or NFA is correct.
  - In particular, each state must have a mnemonic name.
  - For product constructions, explaining the states in the factor DFAs is both necessary and sufficient.
  - Yes, we mean it. A perfectly correct drawing of a perfectly correct DFA with no state explanation is worth at most 6 points.

+ 4 for correctness.
  - -1 for a single mistake: a single misdirected transition, a single missing or extra accepting state, rejecting exactly one string that should be accepted, or accepting exactly one string that should be accepted. (The incorrectly accepted/rejected string is almost always the empty string ε.)
  - -4 for incorrectly accepting every string, or incorrectly rejecting every string.

- -2 for incorrectly accepting/rejecting more than one but a finite number of strings.
- -4 for incorrectly accepting/rejecting an infinite number of strings.

2. Prove that the following languages over the alphabet $\Sigma = \{\mathtt{0}, \mathtt{1}\}$ are not regular by proving that there is an infinite fooling set for each of them.

   (a) $\{\mathtt{0}^a\mathtt{1}\mathtt{0}^b\mathtt{1}\mathtt{0}^c : 2b = a + c\}$.

   (b) The set of all palindromes in Σ^* whose lengths are divisible by 7.

Standard fooling set rubric. 10 points =

+ 4 for the fooling set.
  - $+2$ for explicitly describing the proposed fooling set F.
  - $+2$ if F is truly a fooling set for the lanugage.
  - -4 if F is not a fooling set for the target language.
  - No credit for the *problem* if F is finite.

+ 6 for the proof.
  - The proof must correctly consider arbitrary pairs of distinct strings $x, y \in F$.
  - No credit for the proof unless both x and y are always in F.
  - No credit for the proof unless x and y can be any pair of distinct strings in F.
  - $+2$ for explicitly describing a suffix z that distinguishes x and y.
  - $+2$ for proving either $xz \in L$ or $yz \in L$.
  - $+2$ for proving either $xz \notin L$ or $yz \notin L$, respectively.

Solved problems

4. For each of the following languages, either prove that the language is regular (by constructing an appropriate DFA, NFA, or regular expression) or prove that the language is not regular (by constructing an infinite fooling set).

   Recall that a *palindrome* is a string that equals its own reversal: $w = w^R$. Every string of length 0 or 1 is a palindrome.

   (a) Strings in $(\mathtt{0} + \mathtt{1})^*$ in which no prefix of length at least 2 is a palindrome.

Solution: Regular: $\varepsilon + \mathtt{01}^* + \mathtt{10}^*$. Call this language L_a.

Let w be an arbitrary non-empty string in $(\mathtt{0}+\mathtt{1})^*$. Without loss of generality, assume $w = \mathtt{0}x$ for some string x. There are two cases to consider.

- If x contains a $\mathtt{0}$, then we can write $w = \mathtt{01}^n\mathtt{0}y$ for some integer n and some string y. The prefix $\mathtt{01}^n\mathtt{0}$ is a palindrome of length at least 2. Thus, $w \notin L_a$.
- Otherwise, $x \in \mathtt{1}^*$. Every non-empty prefix of w is equal to $\mathtt{01}^n$ for some non-negative integer $n \le |x|$. Every palindrome that starts with $\mathtt{0}$ also ends with $\mathtt{0}$, so the only palindrome prefixes of w are ε and $\mathtt{0}$, both of which have length less than 2. Thus, $w \in L_a$.

We conclude that $\mathtt{0}x \in L_a$ if and only if $x \in \mathtt{1}^*$. A similar argument implies that $\mathtt{1}x \in L_a$ if and only if $x \in \mathtt{0}^*$. Finally, trivially, $\varepsilon \in L_a$. ■

Rubric: 2½ points = ½ for "regular" + 1 for regular expression + 1 for justification. This is more detail than necessary for full credit.

   (b) Strings in $(\mathtt{0} + \mathtt{1} + \mathtt{2})^*$ in which no prefix of length at least 2 is a palindrome.

Solution: Not regular. Call this language L_b.

Consider the set $F = (\mathtt{012})^+$.

Let x and y be arbitrary distinct strings in F.

Then $x = (\mathtt{012})^i$ and $y = (\mathtt{012})^j$ for some positive integers $i \neq j$.

Without loss of generality, assume $i < j$.

Let z be the suffix $(\mathtt{210})^i$.

- $xz = (\mathtt{012})^i(\mathtt{210})^i$ is a palindrome of length $6i \ge 2$, so $xz \notin L_b$.
- $yz = (\mathtt{012})^j(\mathtt{210})^i$ has no palindrome prefixes except ε and $\mathtt{0}$, because $i < j$, so $yz \in L_b$.

Thus, z is a distinguishing suffix for x and y.

We conclude that F is a fooling set for L_b.

Because F is infinite, L_b cannot be regular. ■

Rubric: 2½ points = ½ for "not regular" + 2 for fooling set proof (standard rubric, scaled).

(c) Strings in $(0+1)^*$ in which no prefix of length at least 3 is a palindrome.

Solution: Not regular. Call this language L_c.
Consider the set $F = (001101)^+$.
Let x and y be arbitrary distinct strings in F.
Then $x = (001101)^i$ and $y = (001101)^j$ for some positive integers $i \neq j$.
Without loss of generality, assume $i < j$.
Let z be the suffix $(101100)^i$.

- $xz = (001101)^i(101100)^i$ is a palindrome of length $12i \geq 2$, so $xz \notin L_b$.
- $yz = (001101)^j(101100)^i$ has no palindrome prefixes except ε and 0 and 00, because $i < j$, so $yz \in L_b$.

Thus, z is a distinguishing suffix for x and y.
We conclude that F is a fooling set for L_c.
Because F is infinite, L_c cannot be regular. ■

Rubric: 2½ points = ½ for "not regular" + 2 for fooling set proof (standard rubric, scaled).

(d) Strings in $(0+1)^*$ in which no *substring* of length at least 3 is a palindrome.

Solution: Regular. Call this language L_d.

Every palindrome of length at least 3 contains a palindrome substring of length 3 or 4. Thus, the complement language $\overline{L_d}$ is described by the regular expression

$$(0+1)^*(000+010+101+111+0110+1001)(0+1)^*$$

Thus, $\overline{L_d}$ is regular, so its complement L_d is also regular. ■

Solution: Regular. Call this language L_d.

In fact, L_d is *finite*! Appending either 0 or 1 to any of the underlined strings creates a palindrome suffix of length 3 or 4.

$$\varepsilon + 0 + 1 + 00 + 01 + 10 + 11 + 001 + \underline{011} + \underline{100} + 110 + \underline{0011} + \underline{1100}$$

■

Rubric: 2½ points = ½ for "regular" + 2 for proof:

- 1 for expression for $\overline{L_d}$ + 1 for applying closure
- 1 for regular expression + 1 for justification